

Graphs: Minimum Spanning Tree and Disjoint Set

A Problem-Solving Companion to Striver's A2Z and SDE Sheets

Maruri Saikiran

July 28, 2026

Contents

1	Graphs: Minimum Spanning Tree and Disjoint Set	1
1.1	The Disjoint Set Data Structure	1
1.2	Disjoint Set (Union-Find)	2
1.3	Kruskal's Algorithm (Minimum Spanning Tree)	3
1.4	Prim's Algorithm (Minimum Spanning Tree)	5
1.5	Number of Provinces	7
1.6	Minimum Cost to Connect All Points	9
1.7	Number of Operations to Make Network Connected	11
1.8	Most Stones Removed with Same Row or Column	13
1.9	Accounts Merge	15
1.10	Number of Islands II	17
1.11	Making a Large Island	19
1.12	Swim in Rising Water	21

Chapter 1

Graphs: Minimum Spanning Tree and Disjoint Set

A spanning tree connects every node using the fewest possible edges, with no cycles. A minimum spanning tree does the same at the lowest possible total weight. Answering “would adding this edge create a cycle” efficiently – without re-running a traversal every time – is what the Disjoint Set data structure is built for, and once it exists, an entire family of “group things by connectivity” problems falls out almost for free.

1.1 The Disjoint Set Data Structure

A **Disjoint Set** (also called **Union-Find**) maintains a collection of nodes partitioned into disjoint groups, supporting two operations efficiently: $find(x)$ (which group is x in?) and $union(x, y)$ (merge x ’s and y ’s groups). Both run in **nearly constant** amortized time with two optimizations:

Path Compression and Union by Size/Rank

- **Path compression:** during $find(x)$, after walking up to the group’s root, point every node visited along the way **directly** to the root – flattening the tree so future lookups for those nodes are immediate.
- **Union by size (or rank):** when merging two groups, attach the **smaller** group’s root under the **larger** group’s root, keeping the resulting tree shallow rather than letting it grow into a long chain.

Together, these two optimizations give nearly $O(1)$ amortized time per operation (formally $O(\alpha(n))$, the inverse Ackermann function, which is ≤ 4 for any practically sized input).

What This Chapter Covers

- **Disjoint Set itself (Section 1.2).**
- **Kruskal’s (Section 1.3) and Prim’s (Section 1.4)** algorithms for building a Minimum Spanning Tree – one sorts edges and uses Disjoint Set to reject cycle-forming edges, the other grows a single tree greedily using a priority queue, mirroring Dijkstra’s frontier expansion.
- **Number of Operations to Make Network Connected (Section 1.7), Most Stones Removed (Section 1.8), Accounts Merge (Section 1.9), Number of Islands II (Section 1.10), and Making a Large Island (Section 1.11):** five problems that are not MST problems at all, but where Disjoint Set’s efficient *grouping* (not minimum-weight connecting) is exactly the tool needed.

1.2 Disjoint Set (Union-Find)

Problem Statement. Implement a data structure supporting: $find(x)$ – determine which group x currently belongs to – and $union(x, y)$ – merge x 's and y 's groups into one, both as efficiently as possible.

Pattern Recognition

“Efficiently track and merge groups of connected elements” is the textbook use case for Disjoint Set: each element points to a **parent**, with a group's identity determined by walking parent pointers up to a **root** (a node that is its own parent). Path compression and union by size, described in Section 1.1, keep both operations nearly constant time.

Algorithm

1. Initialize: every node is its own parent (its own group), and every group has size 1.
2. $find(x)$: if x is its own parent, return x . Otherwise, recursively find x 's ultimate root, and **set x 's parent directly to that root** before returning (path compression).
3. $union(x, y)$: find both roots; if equal, they are already in the same group, do nothing. Otherwise, attach the **smaller**-size root under the **larger**-size root's parent pointer, and add the smaller group's size into the larger's.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class DisjointSet {
5      vector<int> parent, size;
6  public:
7      DisjointSet(int n) : parent(n), size(n, 1) {
8          for (int i = 0; i < n; i++) parent[i] = i;
9      }
10
11     int find(int x) {
12         if (parent[x] == x) return x;
13         return parent[x] = find(parent[x]); // path compression
14     }
15
16     void unite(int x, int y) {
17         int rootX = find(x), rootY = find(y);
18         if (rootX == rootY) return;
19
20         if (size[rootX] < size[rootY]) swap(rootX, rootY);
21         parent[rootY] = rootX; // smaller under larger
22         size[rootX] += size[rootY];
23     }
24
25     bool connected(int x, int y) {
26         return find(x) == find(y);
27     }
28 };
29
30 int main() {

```

```

31     DisjointSet ds(6);
32     ds.unite(0, 1);
33     ds.unite(1, 2);
34     ds.unite(3, 4);
35
36     cout << (ds.connected(0, 2) ? "true" : "false") << endl;    // true
37     cout << (ds.connected(0, 3) ? "true" : "false") << endl;    // false
38     ds.unite(2, 3);
39     cout << (ds.connected(0, 4) ? "true" : "false") << endl;    // true
40     return 0;
41 }

```

Dry run

Dry Run. Take 6 elements, performing *unite*(0,1), *unite*(1,2), *unite*(3,4), then *unite*(2,3).

Initially each node is its own root, size 1 each. *unite*(0,1): both size 1, *rootX* = 0 stays (tie, no swap needed in this implementation since $1 < 1$ is false), *parent*[1] = 0, *size*[0] = 2. *unite*(1,2): *find*(1) = 0 (size 2), *find*(2) = 2 (size 1); 2's group is smaller, *parent*[2] = 0, *size*[0] = 3. *unite*(3,4): similarly, *parent*[4] = 3, *size*[3] = 2. *unite*(2,3): *find*(2) = 0 (size 3, via path compression directly), *find*(3) = 3 (size 2); 3's group is smaller, *parent*[3] = 0, *size*[0] = 5. Now 0, 1, 2, 3, 4 are all connected (root 0); 5 remains its own separate group.

Complexity Analysis

Time Complexity. $O(\alpha(n))$ amortized per *find/union* call, where α is the inverse Ackermann function – effectively constant for any realistic input size. **Space Complexity.** $O(n)$ for the parent and size arrays.

Key Takeaway

Disjoint Set answers “are these two things in the same group, and can I merge their groups” far faster than re-running a DFS/BFS traversal from scratch after every update – making it the tool of choice whenever a problem involves **repeated** connectivity queries or incremental merging, rather than a single one-shot traversal.

1.3 Kruskal’s Algorithm (Minimum Spanning Tree)

Problem Statement. Given a weighted, connected, undirected graph, find a **Minimum Spanning Tree** (MST) – a subset of edges connecting all nodes, with no cycles, at the minimum possible total weight.

Pattern Recognition

“Connect everything as cheaply as possible” is solved greedily: sort all edges by weight, and consider them from **cheapest to most expensive**, adding each edge to the MST unless it would create a **cycle** – checked instantly using Section 1.2’s Disjoint Set (*find* on both endpoints; if equal, they’re already connected, so this edge would close a cycle and is skipped).

Why the greedy choice is correct (the cut property)

For any partition of the nodes into two non-empty groups, the **cheapest** edge crossing between them must belong to *some* MST – if it didn’t, swapping it in for whatever more expensive edge

does cross between the groups in that MST would only decrease the total weight, contradicting minimality. Processing edges from cheapest to most expensive and greedily accepting any edge that connects two still-separate components is exactly repeatedly applying this cut property, which is why the greedy strategy provably produces an MST.

Algorithm

1. Sort all edges by weight, ascending.
2. Initialize a Disjoint Set over all n nodes.
3. For each edge (u, v, w) in sorted order: if $\text{find}(u) \neq \text{find}(v)$, add the edge to the MST (accumulate its weight) and $\text{unite}(u, v)$.
4. Stop once $n-1$ edges have been added (a spanning tree on n nodes always has exactly $n-1$ edges).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class DisjointSet {
5      vector<int> parent, size;
6  public:
7      DisjointSet(int n) : parent(n), size(n, 1) {
8          for (int i = 0; i < n; i++) parent[i] = i;
9      }
10     int find(int x) {
11         if (parent[x] == x) return x;
12         return parent[x] = find(parent[x]);
13     }
14     void unite(int x, int y) {
15         int rootX = find(x), rootY = find(y);
16         if (rootX == rootY) return;
17         if (size[rootX] < size[rootY]) swap(rootX, rootY);
18         parent[rootY] = rootX;
19         size[rootX] += size[rootY];
20     }
21 };
22
23 int kruskalMST(int n, vector<vector<int>>& edges) {
24     sort(edges.begin(), edges.end(),
25          [](const vector<int>& a, const vector<int>& b) { return a[2] <
26                                                         b[2]; });
27
28     DisjointSet ds(n);
29     int totalWeight = 0, edgesUsed = 0;
30
31     for (auto& e : edges) {
32         int u = e[0], v = e[1], w = e[2];
33         if (ds.find(u) != ds.find(v)) {
34             ds.unite(u, v);
35             totalWeight += w;
36             edgesUsed++;
37             if (edgesUsed == n - 1) break;
38         }
39     }
40 }

```

```

38     }
39     return totalWeight;
40 }
41
42 int main() {
43     vector<vector<int>> edges =
44         {{0,1,4},{0,2,1},{1,2,2},{1,3,5},{2,3,8}};
45     cout << kruskalMST(4, edges) << endl; // 8
46 }

```

Dry run

Dry Run. Take edges 0–1(4), 0–2(1), 1–2(2), 1–3(5), 2–3(8), $n = 4$.

Sorted: 0–2(1), 1–2(2), 0–1(4), 1–3(5), 2–3(8). Take 0–2 (weight 1): different components, add; total = 1. Take 1–2 (weight 2): $\text{find}(1) \neq \text{find}(2)$, add; total = 3; now {0, 1, 2} connected. Take 0–1 (weight 4): $\text{find}(0) = \text{find}(1)$ (both already in {0, 1, 2}'s group) – **skip**, would form a cycle. Take 1–3 (weight 5): different components (3 is still separate), add; total = 8. Now 3 edges used (0–2, 1–2, 1–3) = $n-1 = 3$, stop. Final MST weight: 8 – node 3's cheapest available connection in this graph is the weight-5 edge to node 1, since the alternative 2–3 edge costs 8.

Complexity Analysis

Time Complexity. $O(E \log E)$ for sorting, plus $O(E \cdot \alpha(V))$ for the Disjoint Set operations: $O(E \log E)$ overall. **Space Complexity.** $O(V)$ for the Disjoint Set, plus $O(E)$ for sorting the edge list.

Key Takeaway

Kruskal's algorithm is a direct, clean composition of two ideas already established elsewhere in this book series: sorting to consider choices in a greedy-optimal order (the same idea behind every Greedy chapter algorithm), and Disjoint Set to reject cycle-forming choices in near-constant time. Neither idea is new here – their combination is.

1.4 Prim's Algorithm (Minimum Spanning Tree)

Problem Statement. Same as Section 1.3: find a Minimum Spanning Tree of a weighted, connected, undirected graph – this time by **growing a single tree** outward from one starting node, rather than considering all edges globally sorted by weight.

Pattern Recognition

“Grow a tree, always extending it as cheaply as possible” mirrors the Graph Shortest Path chapter's Dijkstra's algorithm almost exactly: a priority queue always offers the **cheapest available edge** connecting some node already in the tree to some node not yet in it. The key difference from Dijkstra: the priority queue tracks the **edge weight to join the tree**, not the **cumulative distance from a source** – a node already close to the tree via one cheap edge is preferred even if a different, longer path from the start node exists.

Why this also satisfies the cut property

At every step, the tree built so far and the remaining nodes form a cut (a partition into two groups). Prim's always picks the **cheapest edge crossing that specific cut** – exactly the

greedy choice the cut property (Section 1.3) guarantees is always safe to include in *some* MST. Kruskal's considers cuts implicitly through global edge sorting; Prim's considers one specific, growing cut at each step.

Algorithm

1. Maintain a *inMST* boolean array and a min-heap of (*weight, node*) pairs, starting with (*0, anyStartNode*).
2. Pop the smallest-weight entry; if its node is already in the MST, skip (stale entry, exactly as in Dijkstra). Otherwise, mark it in the MST and add its weight to the total.
3. For every neighbor of the newly added node not yet in the MST, push (*edgeWeight, neighbor*).
4. Repeat until every node is in the MST.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int primMST(int n, vector<vector<pair<int,int>>>& adj) {
5     vector<bool> inMST(n, false);
6     priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
7     pq.push({0, 0}); // start from node 0, weight 0
8     int totalWeight = 0;
9
10    while (!pq.empty()) {
11        auto [weight, node] = pq.top();
12        pq.pop();
13        if (inMST[node]) continue; // stale entry
14
15        inMST[node] = true;
16        totalWeight += weight;
17
18        for (auto& [neighbor, edgeWeight] : adj[node]) {
19            if (!inMST[neighbor]) {
20                pq.push({edgeWeight, neighbor});
21            }
22        }
23    }
24    return totalWeight;
25 }
26
27 int main() {
28     int n = 4;
29     vector<vector<pair<int,int>>> adj(n);
30     auto addEdge = [&](int u, int v, int w) {
31         adj[u].push_back({v, w});
32         adj[v].push_back({u, w});
33     };
34     addEdge(0, 1, 4); addEdge(0, 2, 1); addEdge(1, 2, 2);
35     addEdge(1, 3, 5); addEdge(2, 3, 8);
36
37     cout << primMST(n, adj) << endl; // 8
38     return 0;
39 }

```

Dry run

Dry Run. Same graph as Section 1.3’s example, starting from node 0.

Heap: [(0,0)]. Pop (0,0): add 0 to MST, total = 0; push neighbors (4,1) and (1,2). Heap: [(1,2), (4,1)]. Pop (1,2) (smallest): add 2, total = 1; push 2’s neighbors not in MST: (2,1) (edge 1–2, weight 2) and (8,3) (edge 2–3, weight 8). Heap: [(2,1), (4,1)_{stale dup}, (8,3)]. Pop (2,1): add 1, total = 3; push 1’s remaining neighbor not in MST: (5,3) (edge 1–3, weight 5). Heap now has both (5,3) and (8,3) for node 3. Pop (4,1)_{stale}: 1 already in MST, skip. Pop (5,3) (smaller than the (8,3) entry): add 3, total = 8. Final MST weight: 8 – matching Kruskal’s result on the same graph, as expected (both algorithms always find a minimum spanning tree, and when the MST is unique, as here, the total weight necessarily agrees).

Complexity Analysis

Time Complexity. $O(E \log V)$ – each edge can trigger one heap push, each costing $O(\log V)$. **Space Complexity.** $O(V)$ for *inMST*, plus $O(E)$ for the heap in the worst case.

Key Takeaway

Kruskal’s and Prim’s solve the identical problem from two different angles – global edge sorting versus local frontier growth – and which is preferable depends on graph density: Kruskal’s $O(E \log E)$ favors sparse graphs (few edges to sort), while Prim’s $O(E \log V)$ with an adjacency-list-based heap favors graphs where V is comparatively small relative to E . Both reuse machinery already built earlier in this book: Kruskal’s reuses Disjoint Set, Prim’s reuses Dijkstra’s exact priority-queue frontier-expansion shape.

1.5 Number of Provinces

Problem Statement. Given an $n \times n$ adjacency matrix *isConnected*, where *isConnected*[*i*][*j*] = 1 means cities *i* and *j* are directly connected, a *province* is a maximal group of directly or indirectly connected cities. Return the number of provinces.

Pattern Recognition

“Count the connected components / groups” is the plainest use of **Disjoint Set**: union every directly connected pair, and the number of provinces is simply the number of **distinct roots** left. (A DFS/BFS component count also works; DSU is shown here because this chapter is about wielding it.)

From matrix to unions

Scan the upper triangle of the matrix. Every 1 at (*i*, *j*) says “*i* and *j* belong to the same province,” which is exactly a *union*(*i*, *j*). After processing all pairs, cities in the same province share one root, so counting how many cities are their own root (*find*(*i*) = *i*) counts the provinces.

Components = Number of Distinct Roots

Disjoint Set never explicitly stores group membership lists; it stores a forest. After all unions, each tree is one component, and each tree has exactly one root. So iterating once and counting self-parents yields the component count in $O(n \alpha(n))$.

Algorithm

1. Create a Disjoint Set over n cities.
2. For every pair $i < j$ with $isConnected[i][j] = 1$, call $unite(i, j)$.
3. Count the indices i with $find(i) = i$; that is the number of provinces.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class DisjointSet {
5      vector<int> parent, size;
6  public:
7      DisjointSet(int n) : parent(n), size(n, 1) {
8          for (int i = 0; i < n; i++) parent[i] = i;
9      }
10     int find(int x) {
11         if (parent[x] == x) return x;
12         return parent[x] = find(parent[x]); // path compression
13     }
14     void unite(int x, int y) {
15         int rx = find(x), ry = find(y);
16         if (rx == ry) return;
17         if (size[rx] < size[ry]) swap(rx, ry);
18         parent[ry] = rx; size[rx] += size[ry]; // smaller under
19         // larger
20     }
21 };
22
23 int numProvinces(vector<vector<int>>& isConnected) {
24     int n = isConnected.size();
25     DisjointSet ds(n);
26     for (int i = 0; i < n; i++)
27         for (int j = i + 1; j < n; j++)
28             if (isConnected[i][j] == 1) ds.unite(i, j);
29
30     int provinces = 0;
31     for (int i = 0; i < n; i++) if (ds.find(i) == i) provinces++;
32     return provinces;
33 }
34
35 int main() {
36     vector<vector<int>> isConnected = {
37         {1, 1, 0},
38         {1, 1, 0},
39         {0, 0, 1}
40     };
41     cout << numProvinces(isConnected) << endl; // 2
42     return 0;
43 }

```

Dry run

Dry Run. The 3×3 matrix above.

- Pair $(0, 1)$ is 1: $\text{unite}(0, 1) - 1$'s root becomes 0.
- Pair $(0, 2)$ and $(1, 2)$ are 0: no union.
- Roots: $\text{find}(0) = 0$, $\text{find}(1) = 0$, $\text{find}(2) = 2$. Self-parents are 0 and 2 – two provinces.

Complexity

Complexity Analysis

Time Complexity. $O(n^2 \alpha(n))$ – scanning the matrix dominates, with near-constant union/find. **Space Complexity.** $O(n)$ for the Disjoint Set arrays.

Key Takeaway

Counting provinces is Disjoint Set at its most direct: union every connected pair, then count distinct roots. Recognize “how many separate groups?” as a component-count question and DSU (or a DFS/BFS flood) answers it immediately.

1.6 Minimum Cost to Connect All Points

Problem Statement. Given n points on a 2D plane, the cost to connect two points is their Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$. Find the minimum total cost to connect all points so that there is exactly one path between any two.

Pattern Recognition

“Connect all nodes at minimum total cost, one path between any two” is the literal definition of a **Minimum Spanning Tree**. The points form a complete graph (every pair is an edge weighted by Manhattan distance); **Kruskal's** algorithm with Disjoint Set selects the cheapest cycle-free edges until all points are joined.

The complete graph and Kruskal's greedy choice

Build every edge (i, j) with weight equal to the Manhattan distance. Kruskal's sorts all edges ascending and adds each one only if its two endpoints are not already connected (checked in near-constant time with Disjoint Set), which is precisely how it avoids cycles. Adding exactly $n - 1$ such edges spans all points at the lowest possible total cost.

Disjoint Set Makes “Would This Edge Form a Cycle?” $O(1)$

Kruskal's correctness rests on always taking the globally cheapest edge that connects two so-far-separate components. Disjoint Set answers “are these endpoints already in the same component?” in $O(\alpha(n))$, turning the cycle check – otherwise an expensive traversal – into an instant lookup.

Algorithm

1. Form all $\binom{n}{2}$ edges (weight, i, j) with weight the Manhattan distance; sort ascending.
2. Create a Disjoint Set over the n points.
3. Scan edges in order; if the endpoints are in different sets, unite them and add the weight to the total. Stop once $n - 1$ edges have been added.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class DisjointSet {
5     vector<int> parent, size;
6 public:
7     DisjointSet(int n) : parent(n), size(n, 1) {
8         for (int i = 0; i < n; i++) parent[i] = i;
9     }
10    int find(int x) {
11        if (parent[x] == x) return x;
12        return parent[x] = find(parent[x]);
13    }
14    bool unite(int x, int y) {
15        int rx = find(x), ry = find(y);
16        if (rx == ry) return false; // already connected
17        if (size[rx] < size[ry]) swap(rx, ry);
18        parent[ry] = rx; size[rx] += size[ry];
19        return true;
20    }
21 };
22
23 int minCostConnectPoints(vector<vector<int>>& points) {
24     int n = points.size();
25     vector<tuple<int,int,int>> edges; // {weight, i, j}
26     for (int i = 0; i < n; i++)
27         for (int j = i + 1; j < n; j++) {
28             int w = abs(points[i][0] - points[j][0]) +
29                 abs(points[i][1] - points[j][1]);
30             edges.push_back({w, i, j});
31         }
32     sort(edges.begin(), edges.end());
33
34     DisjointSet ds(n);
35     int total = 0, used = 0;
36     for (auto& [w, i, j] : edges) {
37         if (ds.unite(i, j)) { // adds only if no
38             // cycle
39             total += w;
40             if (++used == n - 1) break;
41         }
42     }
43     return total;
44 }
45
46 int main() {
47     vector<vector<int>> points = {{0,0},{2,2},{3,10},{5,2},{7,0}};
48     cout << minCostConnectPoints(points) << endl; // 20
49     return 0;
50 }

```

Dry run

Dry Run. Points $\{(0,0), (2,2), (3,10), (5,2), (7,0)\}$.

- Cheapest edges: $(0,0)-(2,2) = 4$, $(2,2)-(5,2) = 3$, $(5,2)-(7,0) = 4$, $(5,2)-(3,10) = 10$.

- Kruskal picks 3 (connect 1, 3), 4 (connect 0, 1), 4 (connect 3, 4), then 10 (connect 2) – four edges for five points.
- Total = 3 + 4 + 4 + 10 = **20**; every later edge would close a cycle and is skipped.

Complexity

Complexity Analysis

Time Complexity. $O(n^2 \log n) - O(n^2)$ edges dominate the sort. **Space Complexity.** $O(n^2)$ for the edge list. (Prim's with an adjacency-free $O(n^2)$ scan uses $O(n)$ space and can be preferable for dense point sets.)

Key Takeaway

“Minimum total cost to connect everything” is always an MST. Modeling the points as a complete graph and running Kruskal's with Disjoint Set reduces the problem to sort-then-union. When the graph is dense (as a complete graph is), consider Prim's $O(n^2)$ variant to avoid materializing every edge.

1.7 Number of Operations to Make Network Connected

Problem Statement. Given n computers and a list of existing cable connections, find the **minimum number of operations** to make every computer connected, where one operation removes a cable and uses it to connect any two currently-disconnected computers. Return -1 if there are not enough cables to possibly connect everything.

Pattern Recognition

“Reuse redundant connections to join separate groups” is a direct Disjoint Set application: union every given connection, then count how many separate **components** remain. Connecting k components into 1 always takes exactly $k-1$ operations (each operation merges two components into one), and a **redundant** cable (one connecting two computers *already* in the same component) is exactly a “spare” cable available to be moved.

Why redundant cables exactly cover the requirement

A graph with n nodes and c connected components needs exactly $c-1$ more edges to become fully connected (this is the same spanning-tree counting argument from Section 1.3: each component is already internally connected, like a single “super-node,” and connecting c super-nodes into one tree needs $c-1$ edges). The number of redundant cables (edges that connected two nodes already in the same component when processed) is exactly $totalCables - (n - c)$, since a spanning structure over n nodes split into c components uses exactly $n - c$ non-redundant edges. So the operation is feasible exactly when the redundant count is $\geq c-1$ – equivalently, exactly when $totalCables \geq n - 1$.

Algorithm

1. If the number of connections is less than $n-1$, return -1 immediately (not enough cable to possibly connect n computers).
2. Union every connection using Disjoint Set.

3. Count the number of distinct components c remaining (count nodes that are their own root).
4. Return $c-1$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class DisjointSet {
5      vector<int> parent, size;
6  public:
7      DisjointSet(int n) : parent(n), size(n, 1) {
8          for (int i = 0; i < n; i++) parent[i] = i;
9      }
10     int find(int x) {
11         if (parent[x] == x) return x;
12         return parent[x] = find(parent[x]);
13     }
14     void unite(int x, int y) {
15         int rootX = find(x), rootY = find(y);
16         if (rootX == rootY) return;
17         if (size[rootX] < size[rootY]) swap(rootX, rootY);
18         parent[rootY] = rootX;
19         size[rootX] += size[rootY];
20     }
21 };
22
23 int makeConnected(int n, vector<vector<int>>& connections) {
24     if ((int)connections.size() < n - 1) return -1;
25
26     DisjointSet ds(n);
27     for (auto& c : connections) ds.unite(c[0], c[1]);
28
29     int components = 0;
30     for (int i = 0; i < n; i++) {
31         if (ds.find(i) == i) components++;
32     }
33     return components - 1;
34 }
35
36 int main() {
37     vector<vector<int>> connections = {{0,1},{0,2},{1,2}};
38     cout << makeConnected(4, connections) << endl; // 1
39     return 0;
40 }

```

Dry run

Dry Run. Take $n = 4$, $connections = [[0, 1], [0, 2], [1, 2]]$.

3 connections $\geq n-1 = 3$: feasible. Union 0–1, 0–2 (now $\{0, 1, 2\}$ connected), 1–2 (redundant – already same component). After processing, components: $\{0, 1, 2\}$ (root, say, 0) and $\{3\}$ (its own root) – $c = 2$. Answer: $c - 1 = 1$ – one redundant cable (the 1–2 connection) can be moved to connect node 3.

Complexity Analysis

Time Complexity. $O((n + E) \cdot \alpha(n))$ – unioning every connection plus a final scan over all nodes. **Space Complexity.** $O(n)$ for the Disjoint Set.

Key Takeaway

This is the first of several problems in this chapter where Disjoint Set is used purely for its **grouping** ability – counting and merging components – with no notion of edge weight or minimality involved at all, distinguishing it clearly from the MST algorithms earlier in this chapter that also relied on Disjoint Set.

1.8 Most Stones Removed with Same Row or Column

Problem Statement. Given the coordinates of n stones on a 2D plane, a stone can be **removed** if some **other**, not-yet-removed stone shares its row or its column. Find the maximum number of stones that can be removed.

Pattern Recognition

“*Stones connected by shared rows/columns*” is a Disjoint Set grouping problem in disguise: union every pair of stones sharing a row or column, then within each resulting connected group, **every stone but one** can always be removed (remove them one at a time, always leaving at least one survivor in the group to justify removing the next). The answer is simply $totalStones - numberOfGroups$.

Why one survivor per group is always achievable

Within a connected group of stones (connected via a chain of shared rows/columns), pick any spanning tree of that connectivity (exactly as in Section 1.3’s spanning tree idea) and remove stones as **leaves** of that tree, working inward: a leaf stone always shares its row or column with its tree-parent, which is guaranteed not yet removed (it’s deeper in the removal order), so the removal condition is always satisfiable until only the root remains. This is why the answer doesn’t depend on *how* stones happen to connect within a group – only on *how many* groups there are.

Building the Disjoint Set over rows and columns

Rather than unioning stones pairwise directly (which would need checking every pair, $O(n^2)$), union each stone’s **row** and **column** together in a combined coordinate space: treat row indices and column indices as Disjoint Set elements in two disjoint ranges (e.g., column c mapped to $c + 10001$ to avoid colliding with row indices). Two stones sharing a row or column end up in the same Disjoint Set group automatically, transitively, without ever comparing stones to each other directly.

C++ implementation

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class DisjointSet {
5     unordered_map<int, int> parent;
6 public:
```

```

7   int find(int x) {
8       if (parent.find(x) == parent.end()) parent[x] = x;
9       if (parent[x] == x) return x;
10      return parent[x] = find(parent[x]);
11  }
12  void unite(int x, int y) {
13      parent[find(x)] = find(y);
14  }
15 };
16
17 int removeStones(vector<vector<int>>& stones) {
18     DisjointSet ds;
19     for (auto& s : stones) {
20         ds.unite(s[0], s[1] + 10001); // column shifted into a separate
                                     range
21     }
22
23     unordered_set<int> roots;
24     for (auto& s : stones) {
25         roots.insert(ds.find(s[0]));
26     }
27     return (int)stones.size() - (int)roots.size();
28 }
29
30 int main() {
31     vector<vector<int>> stones = {{0,0},{0,1},{1,0},{1,2},{2,1},{2,2}};
32     cout << removeStones(stones) << endl; // 5
33     return 0;
34 }

```

Dry run

Dry Run. Take $stones = [(0, 0), (0, 1), (1, 0), (1, 2), (2, 1), (2, 2)]$.

Union (row 0, col 0+10001), (row 0, col 1+10001), (row 1, col 0+10001), (row 1, col 2+10001), (row 2, col 1+10001), (row 2, col 2+10001). Tracing connectivity: stone (0,0) links row 0 and col 10001; stone (1,0) links row 1 and col 10001 – so row 0 and row 1 end up in the *same* group via shared column 0. Continuing, every row and column index here ends up merged into a **single** group (rows 0, 1, 2 and columns 0, 1, 2 all connected through the chain of shared coordinates). All 6 stones' row-roots resolve to this one group: *roots* has size 1. Answer: $6 - 1 = 5$.

Complexity Analysis

Time Complexity. $O(n \cdot \alpha(n))$ – unioning and finding over n stones. **Space Complexity.** $O(n)$ for the Disjoint Set's hash-map-based parent storage (rows and columns are sparse, so a hash map avoids allocating a dense array over the full coordinate range).

Key Takeaway

Mapping two conceptually different categories (rows and columns) into a single Disjoint Set by shifting one category's indices into a disjoint numeric range is a general technique whenever “connected through a shared attribute” needs to be modeled without explicitly enumerating every pairwise connection.

1.9 Accounts Merge

Problem Statement. Given a list of accounts, each formatted as $[name, email_1, email_2, \dots]$, merge any accounts that share **at least one** email in common (the same person may have multiple accounts with overlapping email sets). Return the merged accounts, each with its emails sorted, name listed first.

Pattern Recognition

“Merge groups that share any common element” is Disjoint Set over **account indices**: for every email seen, if it has already been seen belonging to some earlier account, union the current account with that earlier one. Accounts ending up with the same root after all unions belong to the same person and should be merged.

Algorithm

1. Initialize a Disjoint Set over account **indices** (*not* emails – emails are not the nodes here, accounts are).
2. Maintain a hash map from email to the **first** account index seen owning it. For each account i , for each of its emails: if the email has been seen before (owned by account j), *unite*(i, j); otherwise, record i as this email’s owner.
3. Group every email by the **root** of its originating account index.
4. For each group, output $[name, \text{sorted emails}]$ (the name can be taken from any account in the group, since merged accounts are assumed to share the same name).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class DisjointSet {
5      vector<int> parent;
6  public:
7      DisjointSet(int n) : parent(n) {
8          for (int i = 0; i < n; i++) parent[i] = i;
9      }
10     int find(int x) {
11         if (parent[x] == x) return x;
12         return parent[x] = find(parent[x]);
13     }
14     void unite(int x, int y) {
15         parent[find(x)] = find(y);
16     }
17 };
18
19 vector<vector<string>> accountsMerge(vector<vector<string>>& accounts)
20 {
21     int n = accounts.size();
22     DisjointSet ds(n);
23     unordered_map<string, int> emailOwner;
24
25     for (int i = 0; i < n; i++) {
26         for (int j = 1; j < (int)accounts[i].size(); j++) {

```

```

26         string& email = accounts[i][j];
27         if (emailOwner.find(email) != emailOwner.end()) {
28             ds.unite(i, emailOwner[email]);
29         } else {
30             emailOwner[email] = i;
31         }
32     }
33 }
34
35 unordered_map<int, set<string>> grouped;
36 for (auto& [email, ownerIndex] : emailOwner) {
37     grouped[ds.find(ownerIndex)].insert(email);
38 }
39
40 vector<vector<string>> result;
41 for (auto& [root, emails] : grouped) {
42     vector<string> merged;
43     merged.push_back(accounts[root][0]); // name
44     merged.insert(merged.end(), emails.begin(), emails.end());
45     result.push_back(merged);
46 }
47 return result;
48 }
49
50 int main() {
51     vector<vector<string>> accounts = {
52         {"John", "johnsmith@mail.com", "john_newyork@mail.com"},
53         {"John", "johnsmith@mail.com", "john00@mail.com"},
54         {"Mary", "mary@mail.com"}
55     };
56     for (auto& acc : accountsMerge(accounts)) {
57         for (string& s : acc) cout << s << " ";
58         cout << endl;
59     }
60     return 0;
61 }

```

Dry run

Dry Run. Take the three accounts above.

Account 0: email johnsmith@mail.com unseen, owner = 0; email john_newyork@mail.com unseen, owner = 0. Account 1: email johnsmith@mail.com **already owned by account 0** – unite(1,0); email john00@mail.com unseen, owner = 1. Account 2: email mary@mail.com unseen, owner = 2. Grouping by root: accounts 0 and 1 share a root (merged via the shared email), contributing all three of their emails sorted together under name "John"; account 2 stands alone under "Mary". Final: two merged account groups.

Complexity Analysis

Time Complexity. $O(\sum |emails| \cdot \alpha(n) + \sum |emails| \log(\sum |emails|))$ – unioning costs near-constant time per email, and sorting each group's emails (via **set**) contributes the logarithmic factor. **Space Complexity.** $O(\sum |emails|)$ for the email ownership map and grouped output.

Key Takeaway

The subtlety in this problem is recognizing that the Disjoint Set's **nodes** are accounts, not emails – emails are merely the *evidence* used to decide which accounts to union. This same “*the connectivity evidence and the thing being grouped are different*” pattern reappears throughout real-world entity-resolution problems (matching user profiles, deduplicating records) well beyond this specific email-based example.

1.10 Number of Islands II

Problem Statement. Given an $m \times n$ grid initially entirely water, and a sequence of positions where land is added **one cell at a time**, return the number of islands **after each addition**.

Pattern Recognition

“Track connected components as elements are added one at a time” is exactly what Disjoint Set is built for, in contrast to the Graph Basics chapter’s Number of Islands (a single, static, one-shot count via traversal): here, the grid **changes incrementally**, and re-running a full DFS/BFS after every single addition would cost $O(mn)$ per step. Disjoint Set instead updates the island count in near-constant time per addition.

Algorithm

1. Maintain a Disjoint Set over all grid cells (indexed as $r \times \text{cols} + c$), and an *islandCount* starting at 0.
2. For each new land position (r, c) : if it’s already land (a duplicate addition), record the current count unchanged and continue. Otherwise, mark it land and increment *islandCount* by 1 (a brand new, isolated island).
3. Check all 4 neighbors: for each neighbor that is already land and in a **different** Disjoint Set group, union them and **decrement** *islandCount* by 1 (two previously separate islands just merged into one).
4. Record *islandCount* as the answer for this step.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class DisjointSet {
5     vector<int> parent, size;
6 public:
7     DisjointSet(int n) : parent(n), size(n, 1) {
8         for (int i = 0; i < n; i++) parent[i] = i;
9     }
10    int find(int x) {
11        if (parent[x] == x) return x;
12        return parent[x] = find(parent[x]);
13    }
14    void unite(int x, int y) {
15        int rootX = find(x), rootY = find(y);
16        if (rootX == rootY) return;

```

```

17         if (size[rootX] < size[rootY]) swap(rootX, rootY);
18         parent[rootY] = rootX;
19         size[rootX] += size[rootY];
20     }
21 };
22
23 vector<int> numIslands2(int m, int n, vector<vector<int>>& positions) {
24     DisjointSet ds(m * n);
25     vector<vector<bool>> isLand(m, vector<bool>(n, false));
26     int islandCount = 0;
27     vector<int> result;
28
29     int dr[] = {-1, 1, 0, 0};
30     int dc[] = {0, 0, -1, 1};
31
32     for (auto& pos : positions) {
33         int r = pos[0], c = pos[1];
34         if (isLand[r][c]) {
35             result.push_back(islandCount);
36             continue;
37         }
38
39         isLand[r][c] = true;
40         islandCount++;
41
42         for (int dir = 0; dir < 4; dir++) {
43             int nr = r + dr[dir], nc = c + dc[dir];
44             if (nr >= 0 && nr < m && nc >= 0 && nc < n && isLand[nr][nc]) {
45                 int idA = r * n + c, idB = nr * n + nc;
46                 if (ds.find(idA) != ds.find(idB)) {
47                     ds.unite(idA, idB);
48                     islandCount--;
49                 }
50             }
51         }
52         result.push_back(islandCount);
53     }
54     return result;
55 }
56
57 int main() {
58     vector<vector<int>> positions = {{0,0},{0,1},{1,2},{1,0},{0,2}};
59     for (int x : numIslands2(3, 3, positions)) cout << x << " ";
60     cout << endl; // 1 1 2 2 1
61     return 0;
62 }

```

Dry run

Dry Run. Take $m = n = 3$, $positions = [(0,0), (0,1), (1,2), (1,0), (0,2)]$.

Add (0,0): new island, count = 1. No land neighbors. Result: 1. Add (0,1): new island, count = 2; neighbor (0,0) is land, union, count = 1. Result: 1. Add (1,2): new island, count = 2; no land neighbors yet. Result: 2. Add (1,0): new island, count = 3; neighbor (0,0) is land (same group as (0,1)), union, count = 2. Result: 2. Add (0,2): new island, count = 3; neighbor (1,2) is land, union, count = 2; neighbor (0,1) is land and – after the previous union

– in a **different** group from $(0, 2)$'s newly merged group, union again, count = 1. Result: 1. Final sequence: 1, 1, 2, 2, 1 – the last addition, $(0, 2)$, happens to bridge *two* previously separate islands at once, dropping the count by 2 in a single step.

Complexity Analysis

Time Complexity. $O(k \cdot \alpha(mn))$ for k position updates, each doing a constant number of Disjoint Set operations. **Space Complexity.** $O(mn)$ for the Disjoint Set and the *isLand* grid.

Key Takeaway

This problem is the clearest illustration in this chapter of Disjoint Set's core advantage over re-traversal: when connectivity needs to be queried or updated **repeatedly** as a structure changes incrementally, Disjoint Set answers each update in near-constant time, where a fresh DFS/BFS per update would be asymptotically far slower.

1.11 Making a Large Island

Problem Statement. Given an $n \times n$ binary grid, change **at most one** cell from 0 to 1 to maximize the size of the largest resulting island (4-connected group of 1s). Return that maximum possible size.

Pattern Recognition

This closing problem of the chapter combines Disjoint Set's component **sizing** (already used implicitly by union-by-size throughout this chapter) with a final scan: first, group every existing island using Disjoint Set, tracking each group's size; then, for every water cell, compute what size would result from flipping **just that one cell** – the sum of the sizes of its (up to 4) **distinct** neighboring islands, plus 1 for the flipped cell itself.

Why distinctness of neighboring islands matters

A water cell might have two or more 4-directional neighbors that belong to the *same* island (e.g., a water cell with land both above and below, where that land happens to already be connected via some other path around the grid). Counting that island's size twice would overcount. So neighboring islands must be collected into a **set** of distinct roots before summing their sizes – exactly the kind of deduplication Disjoint Set's *find* operation makes trivial (two neighbors are “the same island” precisely when *find* returns the same root for both).

Algorithm

1. Build a Disjoint Set over all n^2 cells; union every land cell with its land neighbors.
2. For every water cell, collect the distinct roots among its up to 4 neighbors that are land; the candidate size if flipped is 1 plus the sum of those distinct roots' group sizes.
3. Track the maximum candidate size across all water cells (and separately, the largest existing island's size, in case the grid has no water cell to flip at all – the answer is then simply n^2).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class DisjointSet {
5     vector<int> parent, size;
6 public:
7     DisjointSet(int n) : parent(n), size(n, 1) {
8         for (int i = 0; i < n; i++) parent[i] = i;
9     }
10    int find(int x) {
11        if (parent[x] == x) return x;
12        return parent[x] = find(parent[x]);
13    }
14    void unite(int x, int y) {
15        int rootX = find(x), rootY = find(y);
16        if (rootX == rootY) return;
17        if (size[rootX] < size[rootY]) swap(rootX, rootY);
18        parent[rootY] = rootX;
19        size[rootX] += size[rootY];
20    }
21    int getSize(int x) { return size[find(x)]; }
22 };
23
24 int largestIsland(vector<vector<int>>& grid) {
25     int n = grid.size();
26     DisjointSet ds(n * n);
27     int dr[] = {-1, 1, 0, 0};
28     int dc[] = {0, 0, -1, 1};
29
30     for (int r = 0; r < n; r++) {
31         for (int c = 0; c < n; c++) {
32             if (grid[r][c] == 1) {
33                 for (int dir = 0; dir < 4; dir++) {
34                     int nr = r + dr[dir], nc = c + dc[dir];
35                     if (nr >= 0 && nr < n && nc >= 0 && nc < n && grid[
36                         nr][nc] == 1) {
37                         ds.unite(r * n + c, nr * n + nc);
38                     }
39                 }
40             }
41         }
42
43         int best = 0;
44         bool hasWater = false;
45
46         for (int r = 0; r < n; r++) {
47             for (int c = 0; c < n; c++) {
48                 if (grid[r][c] == 0) {
49                     hasWater = true;
50                     unordered_set<int> distinctRoots;
51                     for (int dir = 0; dir < 4; dir++) {
52                         int nr = r + dr[dir], nc = c + dc[dir];
53                         if (nr >= 0 && nr < n && nc >= 0 && nc < n && grid[
54                             nr][nc] == 1) {
55                             distinctRoots.insert(ds.find(nr * n + nc));

```

```

56         }
57         int candidate = 1;
58         for (int root : distinctRoots) candidate += ds.getSize(
59             root);
60         best = max(best, candidate);
61     } else {
62         best = max(best, ds.getSize(r * n + c));
63     }
64 }
65 return hasWater ? best : n * n;
66 }
67
68 int main() {
69     vector<vector<int>> grid = {
70         {1, 0},
71         {0, 1}
72     };
73     cout << largestIsland(grid) << endl; // 3
74     return 0;
75 }

```

Dry run

Dry Run. Take the 2×2 grid $[[1, 0], [0, 1]]$ (two separate single-cell islands, diagonally placed).

After building the Disjoint Set: cell (0,0) and cell (1,1) are each their own island of size 1 (they are not 4-directionally adjacent to each other). Checking water cell (0,1): neighbors (0,0) (land, size 1) and (1,1) (land, size 1) – two *distinct* roots, candidate = $1 + 1 + 1 = 3$. Checking water cell (1,0): neighbors (0,0) (land, size 1) and (1,1) (land, size 1) – also distinct, candidate = $1 + 1 + 1 = 3$. Best: 3 – flipping either water cell connects both existing islands into one.

Complexity Analysis

Time Complexity. $O(n^2 \cdot \alpha(n^2))$ – building the Disjoint Set and the final scan both touch every cell a constant number of times. **Space Complexity.** $O(n^2)$ for the Disjoint Set.

Key Takeaway

This chapter closes by combining nearly everything it introduced: Disjoint Set’s grouping (Section 1.2), size tracking (the same union-by-size bookkeeping Kruskal’s and Prim’s relied on internally), and the deduplication that distinguishes a *group* from a list of individual elements (echoing Accounts Merge’s account-grouping and Most Stones Removed’s row/column grouping) – a fitting capstone for a chapter built entirely around one data structure’s surprising range of applications.

1.12 Swim in Rising Water

Problem Statement. Given an $n \times n$ grid where $grid[r][c]$ is the elevation of a cell, at time t the water level is t and you may move (in four directions) into any adjacent cell whose elevation is $\leq t$. Starting at (0,0), find the least time to reach $(n - 1, n - 1)$.

Pattern Recognition

“Least time until a path exists,” where time is a rising threshold, is a connectivity-over-time question, and **Disjoint Set** answers it beautifully: activate cells in increasing elevation order, union each with its already-active neighbors, and the answer is the elevation at which the start and end first become connected.

Turning a threshold into unions over time

At time t , exactly the cells with elevation $\leq t$ are usable. As t rises, cells switch on one by one in increasing-elevation order. When a cell switches on, it may bridge previously separate open regions – a *union* with each adjacent already-active cell. The moment $(0, 0)$ and $(n-1, n-1)$ land in the same set, a fully-submerged-enough path exists, and the elevation of the just-activated cell is the minimum time needed.

Process Cells in Elevation Order

Sorting cells by elevation and activating them in that order means the current cell’s elevation is the current water level t . Because we stop the instant start and end connect, that elevation is exactly the least t for which a path exists – equivalently, the minimum over all paths of the maximum elevation on the path.

Algorithm

1. Collect all cells as $(elevation, r, c)$ and sort ascending.
2. Create a Disjoint Set over the n^2 cells; keep an *active* flag per cell.
3. Activate cells in order. For each activated cell, unite it with every adjacent active cell. After each activation, if $(0, 0)$ and $(n-1, n-1)$ are connected, return the current cell’s elevation.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class DisjointSet {
5     vector<int> parent, size;
6 public:
7     DisjointSet(int n) : parent(n), size(n, 1) {
8         for (int i = 0; i < n; i++) parent[i] = i;
9     }
10    int find(int x) {
11        if (parent[x] == x) return x;
12        return parent[x] = find(parent[x]);
13    }
14    void unite(int x, int y) {
15        int rx = find(x), ry = find(y);
16        if (rx == ry) return;
17        if (size[rx] < size[ry]) swap(rx, ry);
18        parent[ry] = rx; size[rx] += size[ry];
19    }
20    bool connected(int x, int y) { return find(x) == find(y); }
21 };
22
23 int swimInWater(vector<vector<int>>& grid) {

```

```

24     int n = grid.size();
25     vector<tuple<int,int,int>> cells;           // {elevation, r, c}
26     for (int r = 0; r < n; r++)
27         for (int c = 0; c < n; c++)
28             cells.push_back({grid[r][c], r, c});
29     sort(cells.begin(), cells.end());
30
31     DisjointSet ds(n * n);
32     vector<vector<bool>> active(n, vector<bool>(n, false));
33     int dr[] = {-1,1,0,0}, dc[] = {0,0,-1,1};
34
35     for (auto& [elev, r, c] : cells) {
36         active[r][c] = true;
37         for (int k = 0; k < 4; k++) {
38             int nr = r + dr[k], nc = c + dc[k];
39             if (nr < 0 || nr >= n || nc < 0 || nc >= n) continue;
40             if (active[nr][nc]) ds.unite(r * n + c, nr * n + nc);
41         }
42         if (ds.connected(0, n * n - 1)) return elev;    // start & end
43         // joined
44     }
45     return -1;
46 }
47
48 int main() {
49     vector<vector<int>> grid = {{0,2},{1,3}};
50     cout << swimInWater(grid) << endl;           // 3
51 }

```

Dry run

Dry Run. Grid $\begin{smallmatrix} 0 & 2 \\ 1 & 3 \end{smallmatrix}$; start (0,0), end (1,1) with id 3.

- Elevation order: 0 (0,0), 1 (1,0), 2 (0,1), 3 (1,1).
- Activate 0: no active neighbors. Activate 1 (1,0): unite with (0,0). Activate 2 (0,1): unite with (0,0).
- Activate 3 (1,1): unite with active neighbors (0,1) and (1,0) – now (0,0) and (1,1) are connected. Return elevation **3**.

Complexity

Complexity Analysis

Time Complexity. $O(n^2 \log n)$ – sorting the n^2 cells dominates; each activation does $O(1)$ near-constant unions. **Space Complexity.** $O(n^2)$ for the Disjoint Set, the cell list, and the active grid.

Key Takeaway

When the question is “the least threshold at which two points become connected,” add elements in threshold order and use Disjoint Set to detect the moment of connection. This “union over increasing time” technique also solves problems phrased as minimizing the maximum edge on a path – for which a Dijkstra-style “minimize the max” (as in Section ??)

is the alternative.